

Politechnika Wrocławska

Wydział Informatyki i Telekomunikacji

Kierunek: IST

**ZESPOŁOWE PRZEDSIĘWZIĘCIE
INFORMATYCZNE**

AI Present Finder

Dawid Chudzicki

Marcin Dolatowski

Bartosz Gotowski

Szymon Kowaliński

Opiekun pracy

dr inż. Marcin Jodłowiec

WROCŁAW 2025

Spis treści

DOKUMENTACJA PROJEKTOWA	4
1 Wykaz symboli, oznaczeń i akronimów	4
2 Cel i zakres przedsięwzięcia	4
2.1 Wstęp	4
2.2 Cel główny	4
2.3 Zakres projektu	5
2.4 Cele biznesowe i techniczne	5
3 Słownik pojęć	6
4 Stan wiedzy w obszarze przedsięwzięcia	6
5 Założenia wstępne	6
5.1 Założenia technologiczne	6
5.2 Założenia projektowe i ograniczenia	7
6 Analiza wymagań i sposób pracy	7
6.1 Iteracja 1: POC (Proof of Concept)	8
6.2 Iteracja 2: Design	8
6.3 Iteracja 3: MVP	9
6.4 Iteracja 4: Deploy	9
6.5 Organizacja pracy zespołu	10
6.6 Przypadki użycia	10
7 Specyfikacja wymagań na produkt programowy	10
7.1 Wymagania funkcjonalne	11
7.2 Wymagania нефunkcjonalne	11
8 Projekt produktu programowego	11
8.1 Architektura systemu	12
8.2 Poziom 1: Kontekst systemu	12
8.3 Poziom 2: Kontenery	12
8.4 Poziom 3: Komponenty wybranych mikroserwisów	16
8.5 Zastosowane wzorce projektowe	20
8.6 Powiązanie z modelami C4	20
9 Model danych i baza danych	20
9.1 Model konceptualny	20
9.2 Implementacja w DBMS	23
10 Modelowanie behawioralne	23
10.1 Diagram sekwencji	23
10.2 Diagramy stanów i aktywności	26
10.3 BPMN	26

11 Prototypowanie interfejsu	29
11.1 Krok 1: Strona główna (Landing Page)	29
11.2 Krok 2: Formularz wyszukiwania	29
11.3 Krok 3: Wywiad z chatbotem	30
11.4 Krok 4: Lista rekomendacji	31
11.5 Krok 5: Nawigacja i zarządzanie	32
11.6 Panel administracyjny	33
12 Implementacja	34
12.1 Algorytm Rerankingu z Weryfikacją (XAI)	34
12.2 Orkiestracja zdarzeń w architekturze CQRS	34
12.3 Strukturyzacja rozmowy przez Tool Calling	35
12.4 Inżynieria promptów (Prompt Engineering)	36
12.5 Równoległa Agregacja Ofert	38
13 Testy	38
13.1 Planowanie testów	38
13.2 Testy z użytkownikami	38
14 Wyniki i analiza badań	39
14.1 Efektywność czasowa	39
14.2 Jakość rekomendacji	39
14.3 Przykłady rerankingu AI (Explainable AI)	39
14.4 Feedback użytkowników	40
14.5 Stabilność	41

DOKUMENTACJA PROJEKTOWA

1 Wykaz symboli, oznaczeń i akronimów

Skrót	Pełna nazwa	Opis
AI	Artificial Intelligence	Sztuczna inteligencja – technologia wykorzystująca algorytmy uczenia maszynowego.
API	Application Programming Interface	Interfejs programistyczny umożliwiający komunikację między systemami.
UI	User Interface	Interfejs użytkownika.
UX	User Experience	Doświadczenie użytkownika.
VPS	Virtual Private Server	Wirtualny serwer prywatny.
OSINT	Open Source Intelligence	Biały wywiad, pozyskiwanie informacji z ogólnodostępnych źródeł.
LLM	Large Language Model	Duży model językowy (np. GPT-4, Gemini).
SSE	Server-Sent Events	Technologia przesyłania aktualizacji z serwera do klienta.
CQRS	Command Query Responsibility Segregation	Wzorzec architektoniczny rozdzielający odczyt od zapisu.
MVP	Minimum Viable Product	Wersja produktu z minimalną liczbą funkcji wystarczającą do wdrożenia.
POC	Proof of Concept	Dowód koncepcji, prototyp mający na celu weryfikację wykonalności.
SPA	Single Page Application	Aplikacja internetowa działająca w obrębie jednej strony HTML.
DTO	Data Transfer Object	Obiekt służący do przesyłania danych między podsystemami.
XAI	Explainable Artificial Intelligence	Wyjaśnialna sztuczna inteligencja, której decyzje są zrozumiałe dla ludzi.

2 Cel i zakres przedsięwzięcia

2.1 Wstęp

Celem projektu jest opracowanie aplikacji webowej do proponowania spersonalizowanych prezentów dla osób na podstawie podanych linków do ich mediów społecznościowych, a także podanych zainteresowań i cech podczas wywiadu z chatbotem. Na podstawie danych system generuje przykładowe propozycje prezentów, aby następnie wyszukać je w zintegrowanych serwisach sklepowych i zwrócić linki do ofert użytkownikowi.

2.2 Cel główny

Stworzenie użytecznego, szybkiego i bezpiecznego narzędzia wspierającego proces znajdowania prezentu poprzez:

- automatyczne wydobycie i analizę cech osoby na podstawie mediów społecznościowych (OSINT),
- interaktywny chat z agentem AI w celu zebrania dodatkowych informacji,
- generowanie trafnych propozycji prezentów na podstawie uzyskanych danych,
- proponowanie konkretnych ofert produktów ze sklepów internetowych (agregacja).

2.3 Zakres projektu

Co wchodzi w zakres projektu:

- interfejs webowy do wprowadzania linków do publicznych profili społecznościowych i prowadzenia chatu z użytkownikiem,
- moduł parsowania i ekstrakcji jedynie publicznych, jawnych danych z podanych profili,
- generowanie propozycji prezentów dla wybranej osoby,
- propozycja ofert prezentów ze sklepów internetowych (OLX, Allegro, eBay, Amazon),
- filtry wyników (np. po cenie, kategorii).

Co nie wchodzi w zakres projektu:

- automatyczne dokonywanie zakupów ani płatności,
- zbieranie prywatnych danych wymagających logowania,
- integracja OAuth z zewnętrznymi kontami użytkownika,
- generowanie treści graficznych produktów (korzystamy z gotowych zdjęć ofert).

2.4 Cele biznesowe i techniczne

Z biznesowego punktu widzenia projekt ma na celu skrócenie czasu potrzebnego na znalezienie prezentu z kilkunastu godzin do kilku minut oraz zwiększenie trafności upominków. Z technicznego punktu widzenia celem jest weryfikacja możliwości wykorzystania modeli LLM do sterowania procesem wyszukiwania i selekcji ofert (Agentic Commerce) oraz integracja wielu źródeł danych w czasie rzeczywistym.

3 Słownik pojęć

Pojęcie	Opis / Definicja
AI Present Finder	Nazwa projektu — aplikacja webowa wykorzystująca sztuczną inteligencję do proponowania spersonalizowanych prezentów.
Użytkownik	Osoba korzystająca z aplikacji w celu znalezienia pomysłu na prezent dla innej osoby.
Osoba obdarowywana	Osoba, której profil jest analizowany przez system w celu znalezienia prezentu.
Chatbot / Agent AI	Moduł oparty na LLM prowadzący rozmowę z użytkownikiem, zadający pytania o preferencje, okazję i budżet.
Profil społecznościowy	Publicznie dostępna strona użytkownika w mediach społecznościowych (np. Instagram, TikTok).
Ekstrakcja danych	Proces automatycznego pobierania i analizy publicznych informacji z profili.
Propozycja prezentu	Wygenerowany przez AI ogólny pomysł na prezent (np. "Aparat analogowy").
Propozycja oferty	Konkretny link do produktu w sklepie internetowym (np. aukcja na Allegro).
Filtrowanie wyników	Zawężanie listy rekomendacji według kryteriów takich jak cena czy kategoria.
Dane publiczne	Ogólnodostępne informacje w Internecie, niewymagające logowania ani specjalnych uprawnień do odczytu.

4 Stan wiedzy w obszarze przedsięwzięcia

Rynek cyfrowego wsparcia w obdarowywaniu można podzielić na dwa główne segmenty: rozwiązania B2B (Corporate Gifting) oraz B2C (Consumer Chatbots). Rozwiązania B2B (np. Giftpack.ai) oferują wysoką jakość dzięki analizie danych, ale są niedostępne dla klienta indywidualnego. Rozwiązania B2C (np. DreamGift) są łatwo dostępne, ale często opierają się na prostych chatbotach bez kontekstu, co prowadzi do niskiej trafności rekomendacji.

Poniższa tabela przedstawia porównanie AI Present Finder z istniejącymi rozwiązaniami konsumenckimi:

Głównym wyróżnikiem AI Present Finder jest połączenie analizy OSINT (rozwiązującej problem "zimnego startu") z lokalną agregacją ofert (OLX, Allegro), co pozwala na znajdowanie unikatowych prezentów niedostępnych w globalnych katalogach typu Amazon.

5 Założenia wstępne

5.1 Założenia technologiczne

- **Frontend:** React (Single Page Application)
- **Backend:** NestJS (Node.js framework)
- **Baza danych:** PostgreSQL

Rozwiązanie	Źródła danych	Interakcja	Dokładność	Integracja e-commerce
AI Present Finder	Media społecznościowe + chatbot	Szczegółowy wywiad (ok. 20 pytań)	Wysoka	OLX, Amazon, eBay, Allegro
DreamGift	Tylko rozmowa	Krótką (ok. 6 pytań)	Niska/ogólna	Amazon
GiftAssistant	3 pola tekstowe	Brak konwersacji	Niska (4 propozycje)	Amazon
Giftruly	Formularz	Kilka pól wyboru	Podstawowa	Amazon
IntelliGift	Pola tekstowe	Szerokie dane	Średnia	Amazon

Tabela 1: Porównanie rozwiązań rynkowych

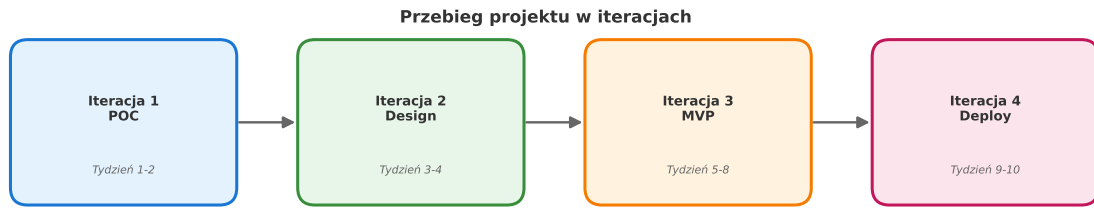
- **Komunikacja:** RabbitMQ (kolejkowanie zadań), REST API, SSE
- **Integracje zewnętrzne:**
 - API mediów społecznościowych (poprzez proxy/scraping)
 - API e-commerce: OLX, Amazon, eBay, Allegro (Sandbox)
 - OpenAI API / Google Gemini API (modele językowe)
- **Hosting:** VPS, obsługa HTTPS

5.2 Założenia projektowe i ograniczenia

- Przetwarzane są wyłącznie dane publicznie dostępne (OSINT).
- System nie obsługuje płatności ani finalizacji transakcji (przekierowanie do sklepu).
- Dane osobowe nie są wykorzystywane w celach marketingowych. Historia wyszukiwań i zapisane produkty są przechowywane wyłącznie dla zalogowanych użytkowników w celu zapewnienia ciągłości sesji.
- Projekt ma charakter edukacyjno-prototypowy (MVP).

6 Analiza wymagań i sposób pracy

Projekt był realizowany iteracyjnie w podejściu zbliżonym do Agile. Zespół korzystał z GitHub Issues jako backlogu produktu, dzieląc prace na kolejne iteracje (POC, Design, MVP, Deploy). Każdy issue reprezentował konkretne wymaganie lub zadanie, a kolumny statusu (Backlog / Todo / Done) odzwierciedlały postęp prac. Przebieg projektu przedstawiono na Rysunku 1.



Rysunek 1: Przebieg projektu w czterech iteracjach.

6.1 Iteracja 1: POC (Proof of Concept)

Celem pierwszej iteracji było sprawdzenie, czy kluczowe ryzyka technologiczne są rozwiązywalne:

- przygotowanie **NestJS boilerplate** z obsługą zdarzeń (issue: "*NestJS boilerplate z zmockowanymi eventami*"),
- **Gift Search POC** – przetestowanie możliwości wyszukiwania prezentów w zewnętrznych serwisach,
- **POC scrapera Facebooka, Instagrama i Tiktoka** – weryfikacja, czy da się zbudować moduł OSINT,
- uruchomienie podstawowego **frontendu** ("set up basic frontend project"),
- przygotowanie pierwszych **diagramów**: C4, BPMN, projekt premise.

Rezultatem iteracji POC było potwierdzenie, że analiza profili, komunikacja zdarzeniowa i podstawowe wyszukiwanie produktów są możliwe w przyjętym stosie technologicznym. Na tym etapie powstały również pierwsze makiety interfejsu (Figma) oraz wstępny podział odpowiedzialności w zespole (backend, frontend, DevOps).

6.2 Iteracja 2: Design

Druga iteracja skupiła się na doprecyzowaniu architektury i modelu domeny:

- utworzenie **diagramów C4** (wersja ogólna i wersja mikroservisowa),
- przygotowanie **diagramu klas** oraz **diagramu BPMN** opisującego przepływ procesu,
- doprecyzowanie podziału na mikroservisy i kontraktów między nimi,
- konfiguracja środowiska developerskiego i CI.

Na koniec iteracji Design powstał spójny model architektoniczny, który został wykorzystany przy budowie MVP.

6.3 Iteracja 3: MVP

Trzecia iteracja miała na celu dostarczenie **działającego MVP** aplikacji:

- wyodrębnienie mikroserwisów: **gift-ideas-microservice**, **reranking-microservice**, **fetch-microservice**,
- integracja chatu z architekturą zdarzeniową ("EPIC: integrate chat POC with microservices setup"),
- dodanie widoków **frontendowych**: home, stalking, chat, rekomendacje,
- implementacja **historii czatu** oraz **historii rekomendacji prezentów**,
- dodanie **ulubionych produktów**, filtrowania i sortowania,
- pierwsza wersja **systemu rerankingu** i integracja z platformami e-commerce.

W trakcie iteracji MVP powstał też **landing page** oraz logotyp projektu. Celem biznesowym było udostępnienie wersji, która rozwiązuje podstawowy problem użytkownika: wygenerowanie sensownych propozycji prezentów w jednym miejscu.

Przykładowo, podczas jednego z wewnętrznych testów MVP okazało się, że standardowe wyszukiwanie produktów zwraca wiele duplikatów oraz ofert zupełnie niepasujących do profilu obdarowywanej osoby (np. części zamiennie zamiast gotowych prezentów). To konkretne doświadczenie doprowadziło do zaprojektowania dedykowanego modułu rerankingu opartego na AI, który odfiltrowuje szum i eksponuje najbardziej trafne propozycje.

6.4 Iteracja 4: Deploy

Ostatnia iteracja skupiła się na przygotowaniu systemu do beta-testów i wdrożenia produkcyjnego:

- dodanie **logowania i autoryzacji** (Google OAuth, uprawnienia, odświeżanie tokenów),
- rozbudowa **panelu użytkownika** oraz **panelu administratora**,
- dopracowanie **UX/UI** (animacje chatu, lepszy layout na desktopie, landing page),
- optymalizacje **rerankingu** (usuwanie duplikatów, pararelizacja oceniania, pętla regeneracji propozycji),
- **tłumaczenie** aplikacji na język polski,
- przygotowanie **deploymentu** (CI/CD, migracje bazy danych, health-checki).

W tej iteracji skoncentrowano się również na przygotowaniu aplikacji do testów z realnymi użytkownikami oraz zbieraniu feedbacku (gwiazdki, formularze opinii). Pojawiły się też mechanizmy refinementu (dopytywanie o lepsze prezenty na podstawie wcześniejszych wyborów) oraz pytanie o budżet przed rozpoczęciem wywiadu.

Na podstawie tak zebranych i iteracyjnie doprecyzowywanych wymagań przygotowano poniższą specyfikację produktu programowego.

6.5 Organizacja pracy zespołu

Zespół pracował w składzie czteroosobowym, z wyraźnym podziałem ról:

- Dawid Chudzicki – lider backendu i Project Manager, odpowiedzialny za architekturę (C4, diagramy sekwencji), logikę rdzeniową oraz moduł autentykacji (Google OAuth).
- Marcin Dolatowski – odpowiedzialny za integracje zewnętrzne (fetch-microservice, sklepy), Reranking Microservice oraz refaktoryzację i dopracowanie interfejsu użytkownika (landing page, UI).
- Bartosz Gotowski – DevOps i infrastruktura (Coolify, Docker, CI/CD, migracje bazy danych), monitoring oraz dokumentacja techniczna (LaTeX, diagramy C4/BPMN).
- Szymon Kowaliński – architektura mikroservisów, implementacja SSE, logika chatu (gotowe odpowiedzi), poprawki i optymalizacje na styku frontend–backend.

Postęp prac był regularnie raportowany prowadzącemu w formie statusów (co kto aktualnie robi, jaki jest plan na sprint i jakie pojawiły się ryzyka), co ułatwiało korygowanie zakresu MVP oraz planowanie kolejnych iteracji (m.in. sprint poświęcony testom i optymalizacji aplikacji).

6.6 Przypadki użycia

Na podstawie zebranych wymagań zdefiniowano kluczowe przypadki użycia (Use Case), m.in.:

- U1: Użytkownik podaje linki do profili społecznościowych osoby obdarowywanej.
- U2: Użytkownik przeprowadza wywiad z chatbotem (okazja, budżet, preferencje).
- U3: System analizuje dane (OSINT + wywiad) i generuje listę pomysłów na prezenty.
- U4: System wyszukuje i prezentuje oferty produktów z wielu sklepów.
- U5: Użytkownik filtruje i sortuje wyniki, dodaje produkty do ulubionych i historii.
- U6: Administrator przegląda statystyki i zarządza użytkownikami (panel admina).

Graficzna reprezentacja tych przypadków została przygotowana w postaci diagramu przypadków użycia UML (Rysunek 2), gdzie aktorami są m.in. *Użytkownik*, *Administrator* oraz zewnętrzne systemy (sklepy, media społecznościowe).

Zidentyfikowane przypadki użycia oraz wymagania zebrane w trakcie iteracji stanowią fundament dla formalnej specyfikacji wymagań, którą przedstawiono w kolejnym rozdziale.

7 Specyfikacja wymagań na produkt programowy

Poniższa specyfikacja formalizuje wymagania zidentyfikowane w trakcie kolejnych iteracji i pracy z backlogiem, tak aby można je było jednoznacznie odwzorować w architekturze i implementacji.

7.1 Wymagania funkcjonalne

- **Wprowadzanie danych:** Możliwość podania linków do profili społecznościowych (Instagram, TikTok).
- **Ekstrakcja danych:** Automatyczne pobranie i analiza publicznych danych z podanych profili.
- **Konwersacja:** Interaktywny chat z botem dopytującym o szczegóły (okazja, budżet, relacja).
- **Analiza:** Łączenie danych z wywiadu i OSINT w spójny profil psychometryczny.
- **Generowanie pomysłów:** Stworzenie listy 5–10 abstrakcyjnych pomysłów na prezent.
- **Wyszukiwanie ofert:** Pobieranie konkretnych ofert z platform e-commerce (OLX, Allegro, Amazon, eBay).
- **Filtrowanie:** Możliwość sortowania i filtrowania wyników (np. po cenie).

7.2 Wymagania niefunkcjonalne

- **Dostępność:** Aplikacja działa w przeglądarce internetowej bez konieczności instalacji.
- **Wydajność:** Czas generowania wstępnych propozycji nie powinien przekraczać 10 sekund (dla chatu), a pełne wyszukiwanie ofert powinno odbywać się w tle z podglądem postępu.
- **Bezpieczeństwo:** Komunikacja szyfrowana protokołem HTTPS.
- **Skalowalność:** Architektura mikroservisowa umożliwiająca łatwe dodawanie nowych źródeł danych.
- **Obsługa błędów:** Odporność na limity API zewnętrznych serwisów (mechanizmy ponawiania).

Powyższy zestaw wymagań funkcjonalnych i niefunkcjonalnych stał się podstawą do opracowania architektury systemu, opisaną szczegółowo w następnej sekcji.

8 Projekt produktu programowego

Zdefiniowane wymagania funkcjonalne i niefunkcjonalne zostały odwzorowane w poniższej architekturze mikroservisowej, obejmującej zarówno warstwę frontendową, jak i zestaw współpracujących usług backendowych.

8.1 Architektura systemu

System został zaprojektowany w architekturze mikroserwisowej, co zapewnia skalowalność i separację odpowiedzialności. Komunikacja między serwisami odbywa się asynchronicznie za pośrednictwem kolejki wiadomości RabbitMQ.

Główne komponenty systemu:

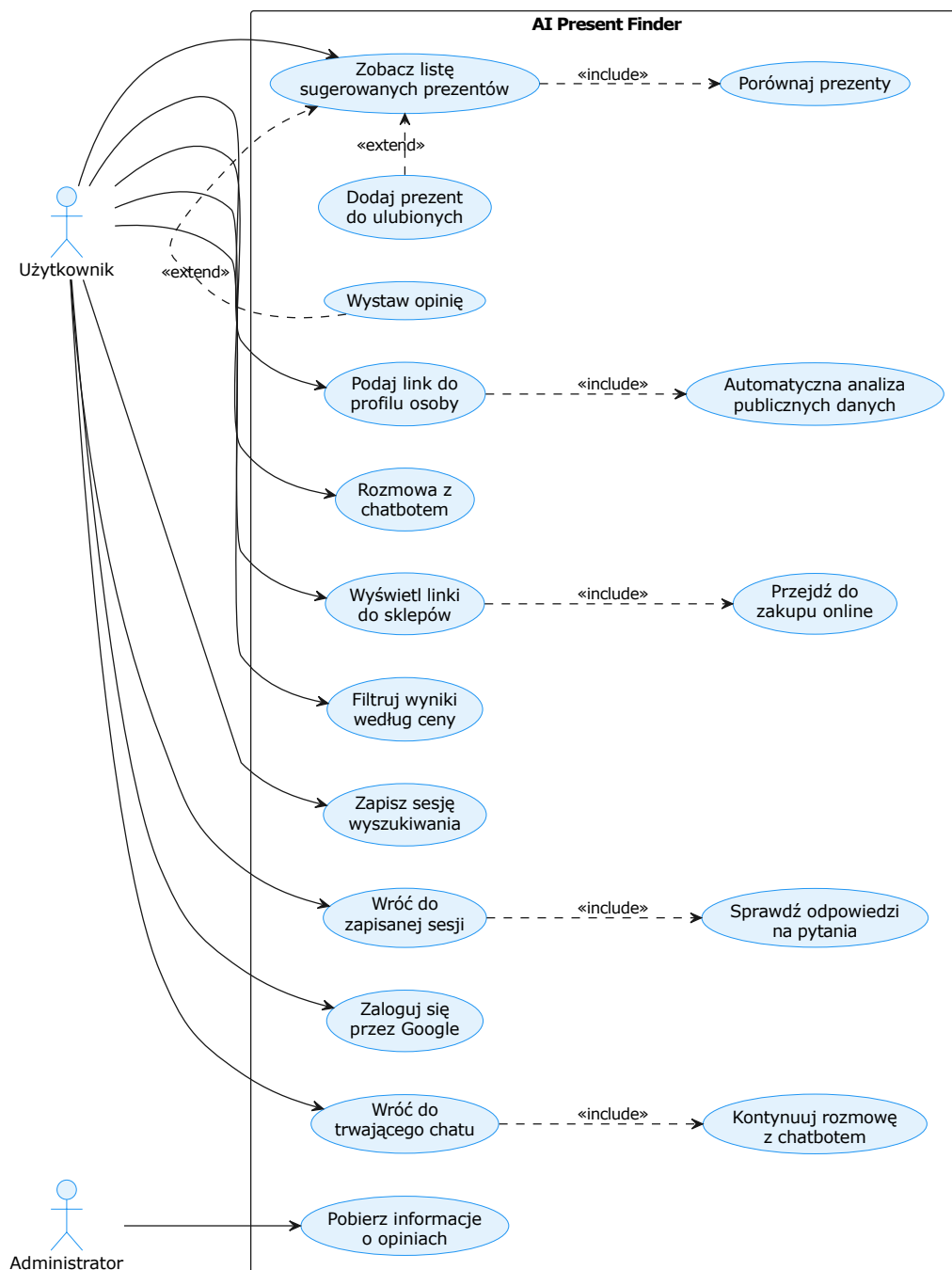
- **Frontend (React):** Interfejs użytkownika komunikujący się z backendem przez REST API oraz odbierający aktualizacje w czasie rzeczywistym przez SSE (Server-Sent Events).
- **RestAPI Macroservice (restapi-macroservice):** Brama wejściowa (API Gateway), obsługująca żądania HTTP i przekierowująca je do odpowiednich kolejek.
- **Chat Microservice (chat-microservice):** Odpowiada za prowadzenie wywiadu z użytkownikiem.
- **Stalking Microservice (stalking-microservice):** Realizuje proces OSINT (analiza profili społecznościowych).
- **Gift Ideas Microservice (gift-ideas-microservice):** Generuje abstrakcyjne pomysły na prezenty na podstawie zebranych danych.
- **Fetch Microservice (fetch-microservice):** Agreguje oferty z zewnętrznych platform (OLX, Allegro, Amazon, eBay). Działa w wielu instancjach równolegle.
- **Reranking Microservice (reranking-microservice):** Ocenia i filtruje znalezione oferty przy użyciu AI.

8.2 Poziom 1: Kontekst systemu

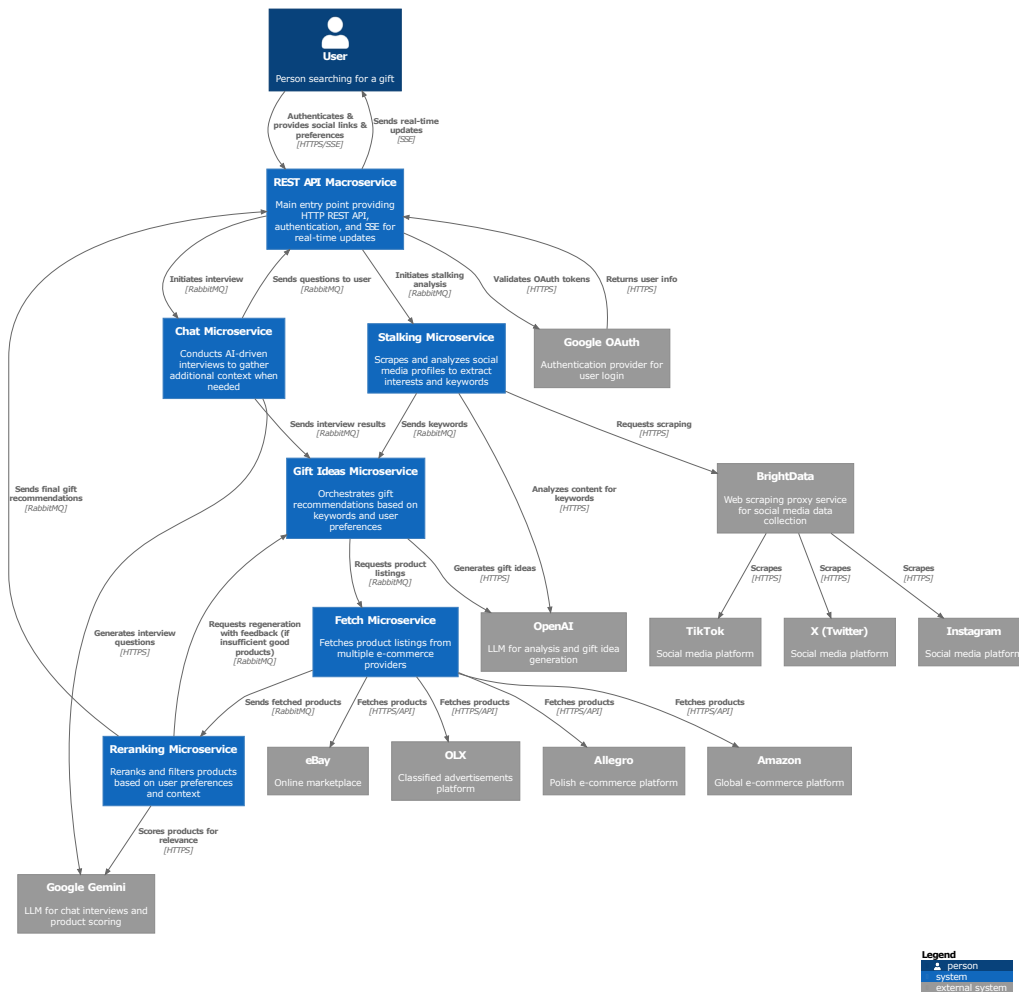
Na najwyższym poziomie abstrakcji (C4 Context) system AI Present Finder jest postrzegany jako pojedynczy blok współpracujący z aktorami zewnętrznymi: użytkownikiem szukającym prezentu, platformami e-commerce (OLX, Allegro, Amazon, eBay), mediami społecznościowymi (Instagram, TikTok) oraz dostawcami modeli LLM (OpenAI, Google). Zależności te przedstawiono na Rysunku 3.

8.3 Poziom 2: Kontenery

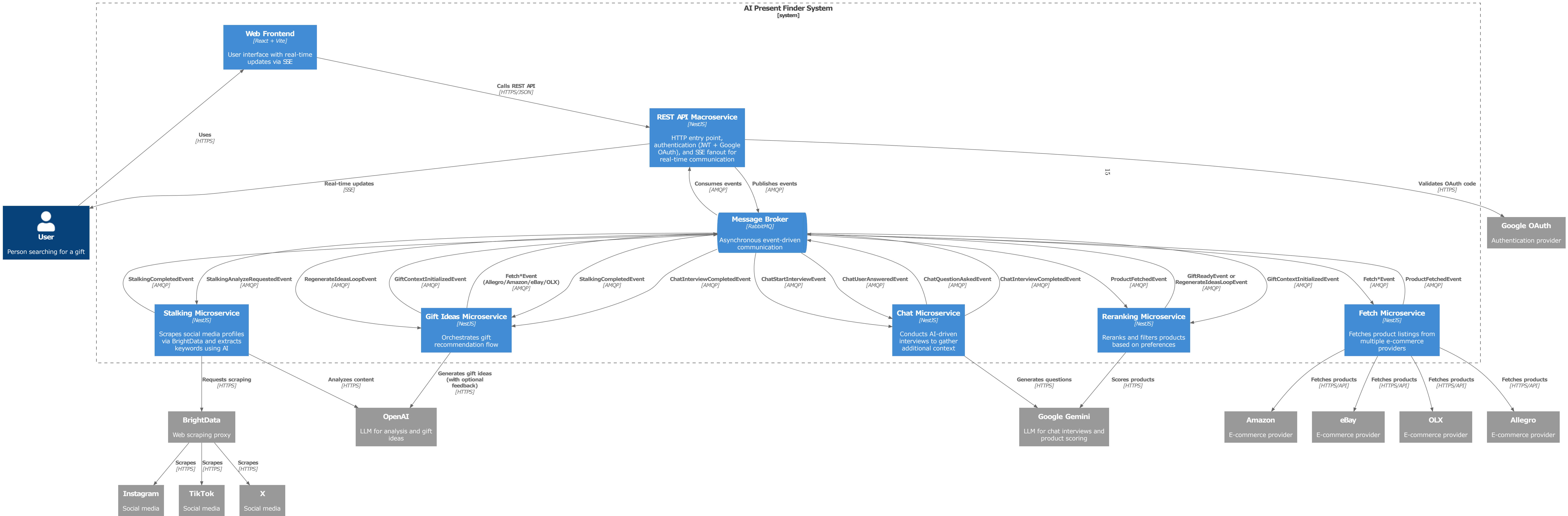
Po zejściu o jeden poziom niżej (C4 Container) widać podział systemu na odrębne jednostki wdrożeniowe: aplikację frontendową (React SPA), bramę API (RestAPI Macroservice), zestaw mikroserwisów backendowych, bazę danych PostgreSQL oraz broker wiadomości RabbitMQ. Każdy kontener może być niezależnie skalowany i wdrażany w środowisku Docker/Coolify. Architekturę kontenerów przedstawiono na Rysunku 8.3.



Rysunek 2: Diagram przypadków użycia systemu AI Present Finder.



Rysunek 3: Diagram kontekstowy systemu AI Present Finder (C4 Context).



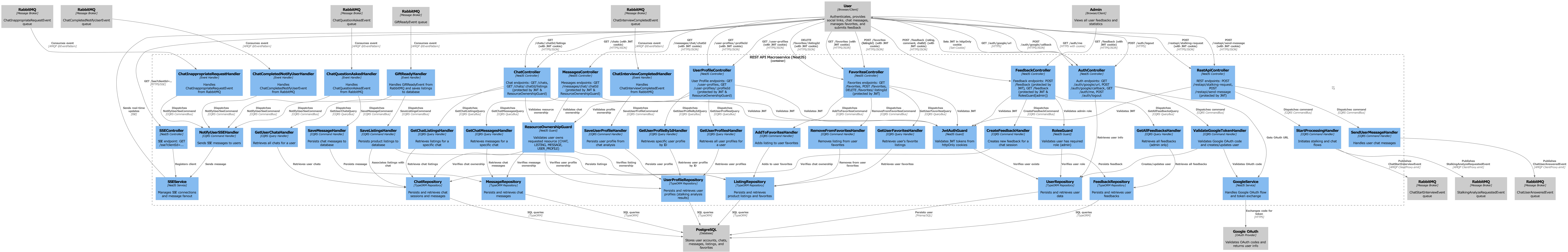
Legend

- person
- container
- external system
- system boundary

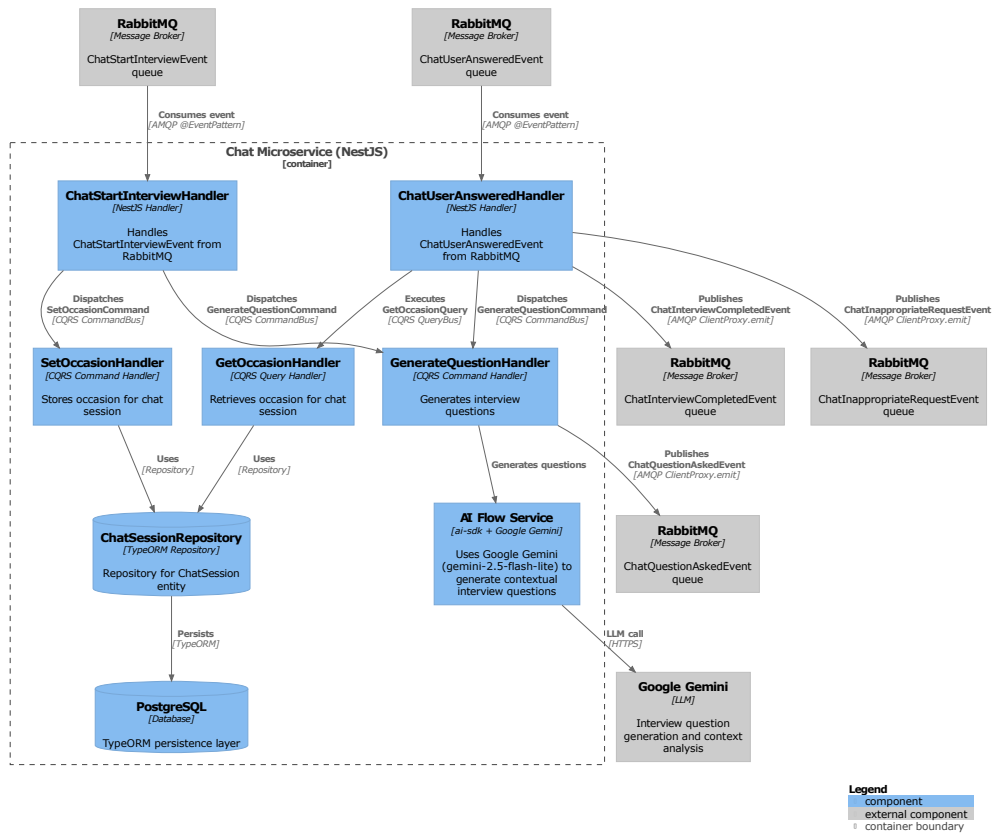
8.4 Poziom 3: Komponenty wybranych mikroservisów

Na najniższym prezentowanym poziomie (C4 Component) każdy mikroservis jest rozbity na wewnętrzne moduły. Poniżej przedstawiono trzy kluczowe serwisy.

RestAPI Macroservice. Brama wejściowa systemu (Rysunek 8.4) składa się z warstwy kontrolerów HTTP, handlerów komend CQRS oraz adapterów publikujących zdarzenia do kolejek RabbitMQ. Odpowiada również za obsługę SSE i autoryzację (Google OAuth).

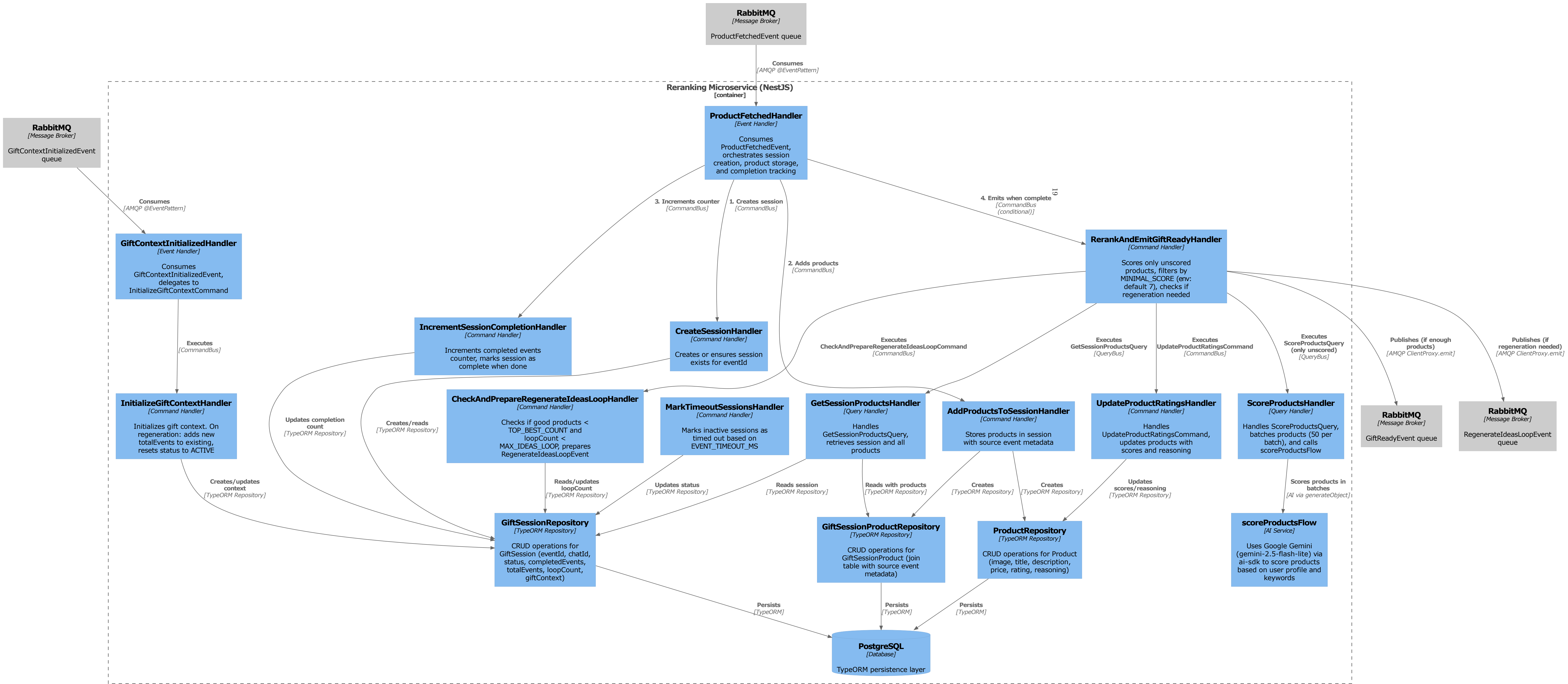


Chat Microservice. Serwis prowadzący wywiad z użytkownikiem (Rysunek 6) wykorzystuje mechanizm *tool calling*, aby wymusić strukturyzowane odpowiedzi od modelu LLM. Wewnętrznie dzieli się na handler zdarzeń, flow AI oraz repozytorium sesji.



Rysunek 6: Diagram komponentów Chat Microservice (C4 Component).

Reranking Microservice. Moduł odpowiedzialny za ocenę i filtrowanie ofert (Rysunek 5) korzysta z modelu Gemini 2.5 do semantycznej analizy każdego produktu. Generuje ocenę punktową oraz tekstowe uzasadnienie, co zapewnia wyjaśnialność decyzji (XAI).



8.5 Zastosowane wzorce projektowe

- **CQRS (Command Query Responsibility Segregation):** Rozdzielenie operacji modyfikujących stan (Command) od zapytań o dane (Query).
- **Event-Driven Architecture:** System reaguje na zdarzenia (np. `StalkingCompletedEvent`, `ProductFetchedEvent`), co pozwala na luźne powiązanie komponentów.
- **Asynchronous Request-Reply:** Obsługa długotrwałych procesów bez blokowania wątku głównego.
- **Strategy:** Wybór odpowiedniego algorytmu pobierania danych w zależności od źródła (np. strategia dla OLX vs strategia dla Amazon).
- **DTO (Data Transfer Object):** Ustrukturyzowane obiekty do przesyłania danych między warstwami.

8.6 Powiązanie z modelami C4

Architektura systemu została zaprojektowana zgodnie z podejściem C4 i odwzorowana na trzech pierwszych poziomach:

- **Poziom 1 – Context:** AI Present Finder jako system wspierający proces wyboru prezentu, współpracujący z użytkownikiem, promotorem (w roli interesariusza) oraz zewnętrznymi dostawcami (sklepy, media społecznościowe, dostawcy LLM).
- **Poziom 2 – Container:** Podział systemu na frontend (przeglądarka), backend (RestAPI, mikroserwisy), bazę danych oraz RabbitMQ. Każdy z mikroservisów jest osobnym kontenerem uruchamianym w środowisku Docker/Coolify.
- **Poziom 3 – Component:** W ramach każdego mikroservisów wyróżniono komponenty takie jak kontrolery HTTP, handlerzy zdarzeń, serwisy domenowe oraz warstwę dostępu do danych (repozytoria TypeORM).

Diagramy C4 (context, container, component) zostały przygotowane w narzędziu PlantUML i są spójne z opisem wymagań oraz podziałem na mikroserwisy przedstawionym w tej sekcji.

Uzupełnieniem projektu architektonicznego jest model danych, który definiuje strukturę przechowywanych informacji i relacje między nimi.

9 Model danych i baza danych

Aby wesprzeć zdefiniowane przypadki użycia i przyjętą architekturę mikroservisową, zaprojektowano model danych odzwierciedlający kluczowe byty domenowe oraz ich relacje.

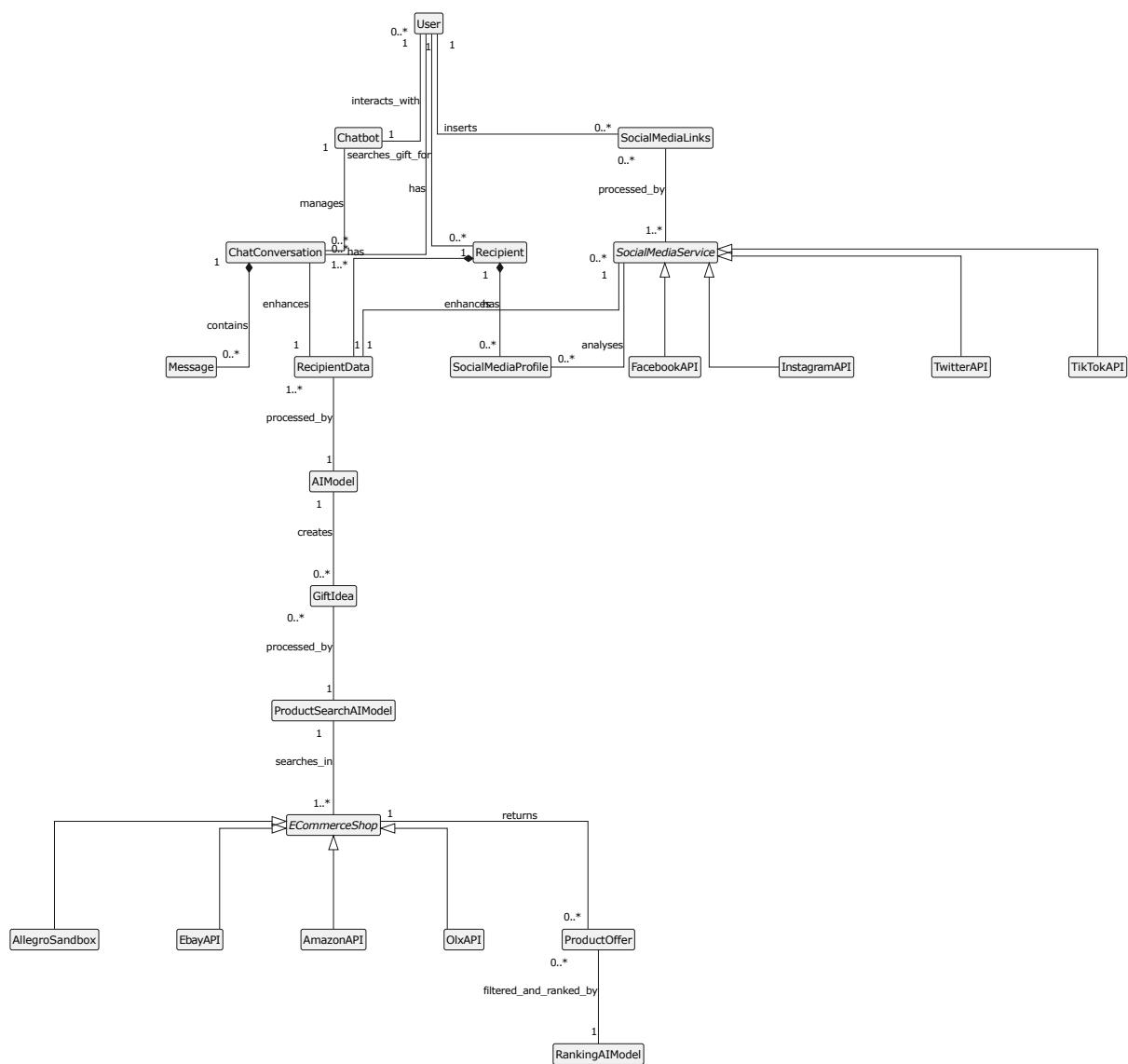
9.1 Model konceptualny

Model danych został zaprojektowany w sposób domenowy, z wykorzystaniem diagramów E/R oraz UML (diagram klas). Główne byty to m.in.:

- **User** – reprezentuje zalogowanego użytkownika systemu (dane z Google OAuth, preferencje),

- **Session** – pojedyncza sesja wyszukiwania prezentów, powiązana z użytkownikiem,
- **ChatMessage** – wiadomości w ramach sesji (pytania i odpowiedzi),
- **GiftIdea** – abstrakcyjny pomysł na prezent wygenerowany przez moduł AI,
- **Product** – konkretna oferta produktu pobrana z serwisu e-commerce,
- **Feedback** – ocena użytkownika dotycząca trafności rekomendacji,
- **Favorite** – relacja użytkownika do wybranych produktów (lista ulubionych).

Relacje między tymi bytami zostały odwzorowane na diagramie koncepcyjnym (User posiada wiele Session; Session posiada wiele ChatMessage i GiftIdea; GiftIdea powiązana jest z wieloma Product; User może dodawać Product do Favorites oraz pozostawiać Feedback dla Session). Na Rysunku 8 przedstawiono diagram klas UML modelu domenowego, natomiast fizyczną strukturę tabel w bazie PostgreSQL pokazuje diagram E/R na Rysunku 9.



Rysunek 8: Diagram klas UML modelu domenowego systemu.

9.2 Implementacja w DBMS

Implementację modelu danych wykonano w PostgreSQL z wykorzystaniem TypeORM. Dla każdej encji domenowej przygotowano odpowiadającą jej tabelę, z kluczami głównymi typu UUID oraz indeksami przyspieszającymi wyszukiwanie po polach takich jak `userId`, `sessionId` czy `provider` (OLX, Allegro, eBay, Amazon).

Początkowo używano trybu `synchronize: true`, jednak na etapie wdrożenia produkcyjnego przełączono się na migracje bazodanowe, aby zapewnić kontrolę nad zmianami schematu. Migracje generowano narzędziem CLI TypeORM i uruchamiano w pipeline CI/CD oraz na serwerze produkcyjnym.

W dokumentacji bazy danych wyróżniono widok całościowy (diagram wszystkich tabel) oraz zbliżenia na kluczowe fragmenty, takie jak model sesji i historii wyszukiwań czy model przechowywania feedbacku użytkowników.

Zdefiniowany model danych stanowi statyczny obraz systemu. Aby w pełni zrozumieć jego działanie, w kolejnym rozdziale przedstawiono modelowanie behawioralne, opisujące dynamikę interakcji i przepływów.

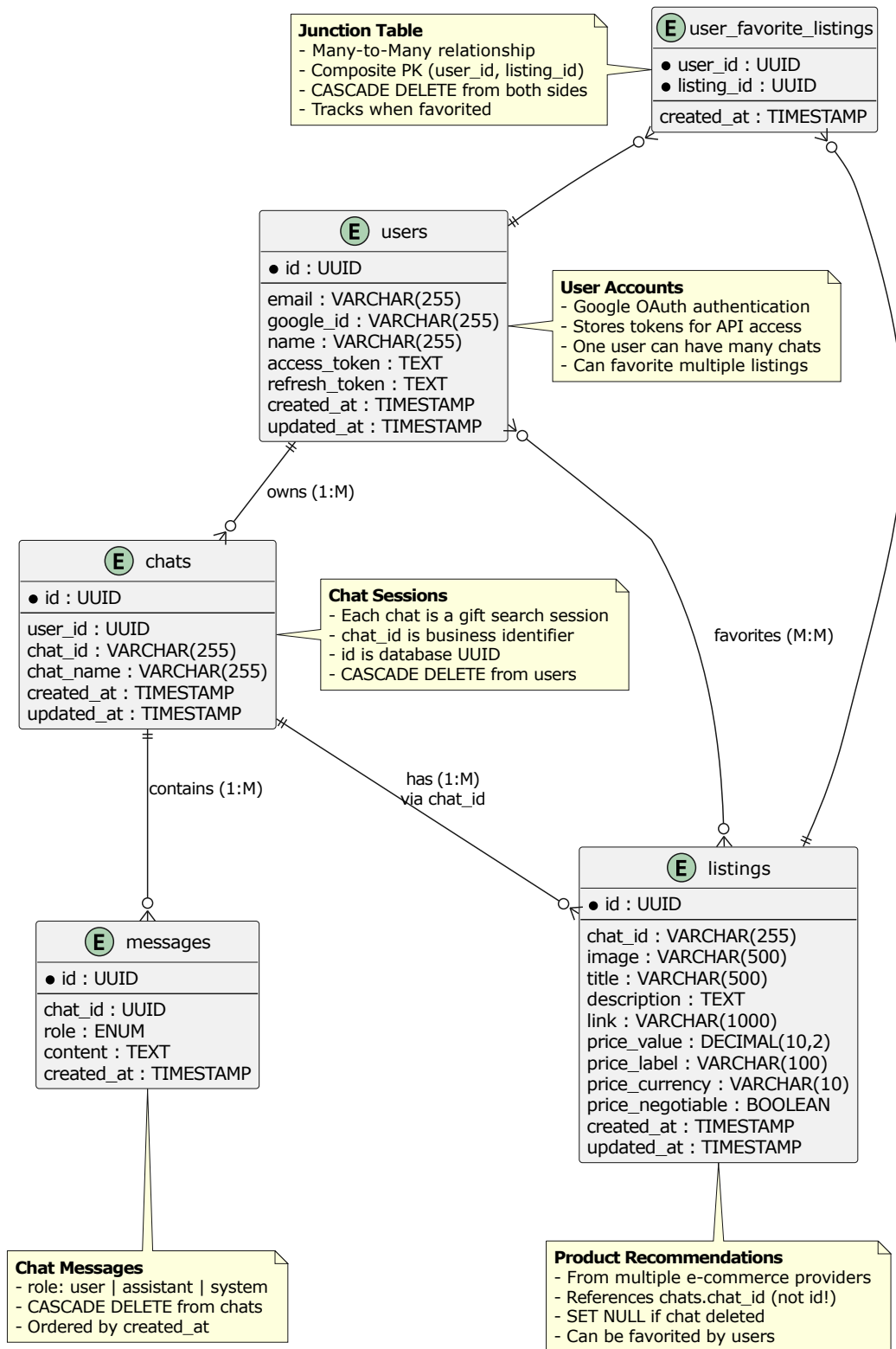
10 Modelowanie behawioralne

Po ustaleniu struktury systemu kluczowe było opisanie jego zachowania w czasie w postaci sekwencji interakcji, stanów oraz procesów biznesowych, które prowadzą użytkownika od podania linku aż do listy konkretnych ofert prezentów.

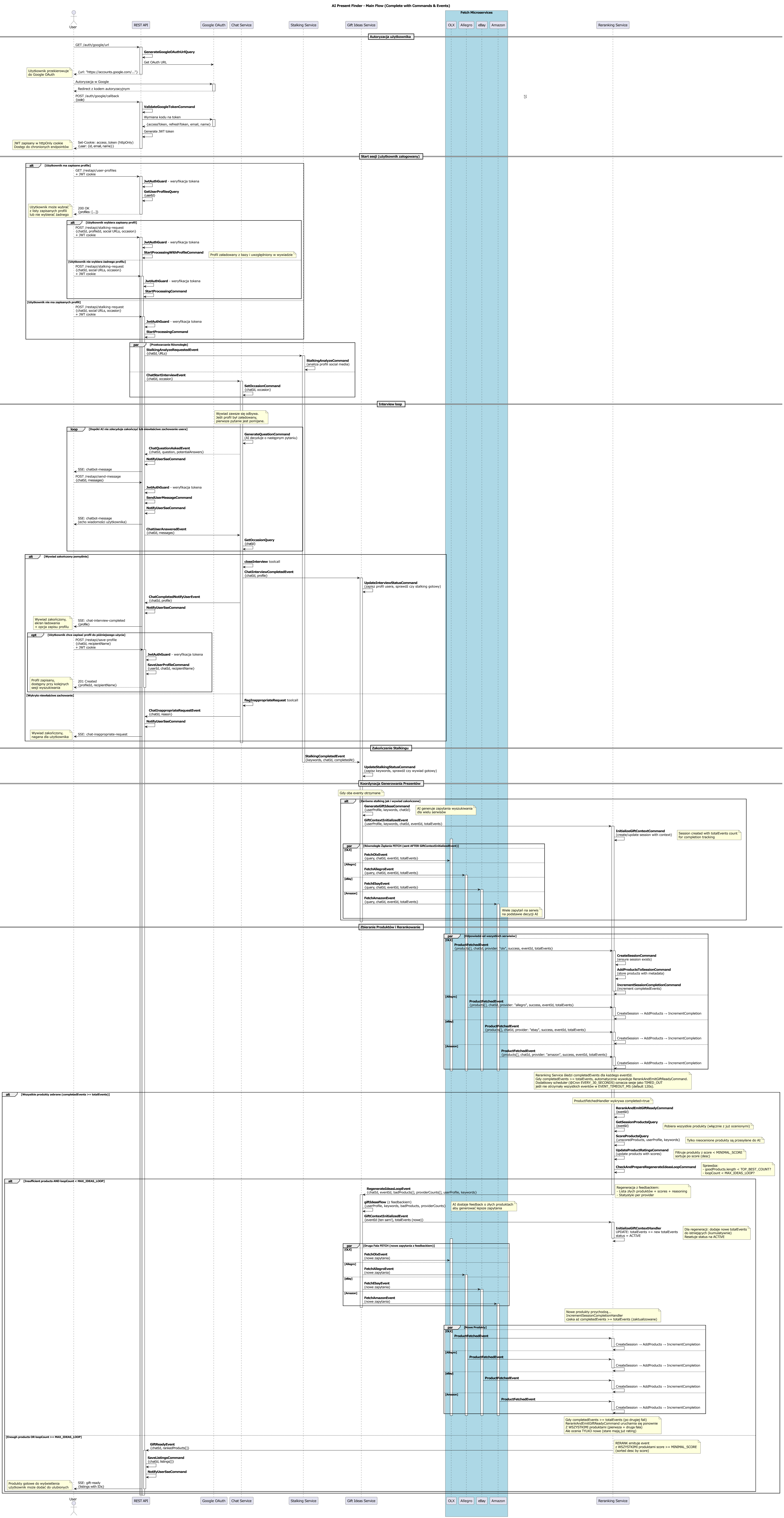
10.1 Diagram sekwencji

Dla głównego scenariusza użycia (wyszukiwanie prezentu) przygotowano diagram sekwencji UML (Rysunek 10.1) przedstawiający przepływ wiadomości pomiędzy: Użytkownikiem, Frontendem, RestAPI Macroservice, Chat Microservice, Stalking Microservice, Gift Ideas Microservice, Fetch Microservice oraz Reranking Microservice. Diagram pokazuje m.in. moment, w którym Chat Microservice decyduje o rozpoczęciu wyszukiwania oraz sposób, w jaki wyniki są strumieniowane do użytkownika przez SSE.

REST API Database - Entity Relationship Diagram



Rysunek 9: Diagram E/R modelu danych głównej bazy (RestAPI).

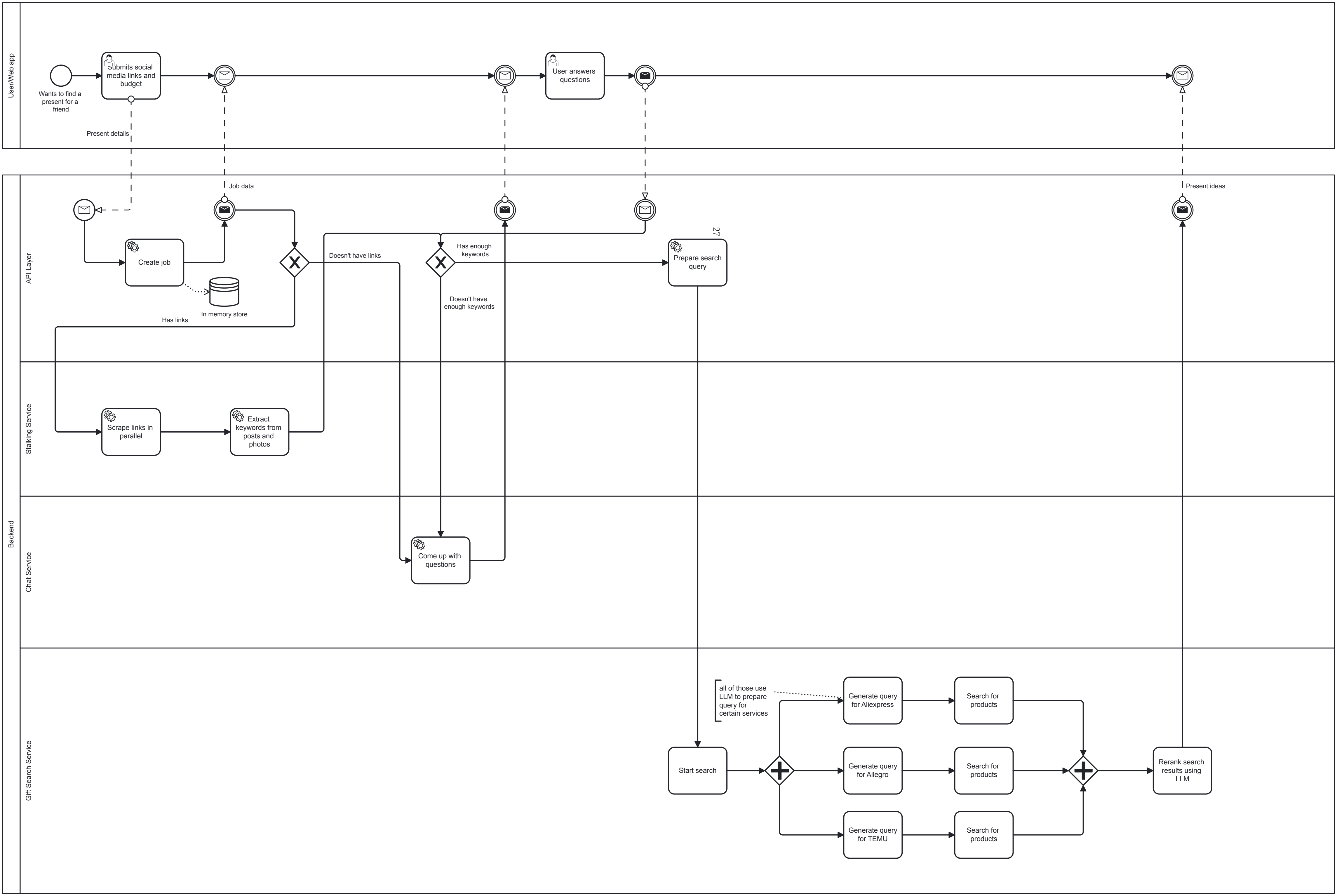


10.2 Diagramy stanów i aktywności

Stan aplikacji z perspektywy sesji użytkownika został opisany za pomocą diagramu stanów (stany: *Nowa*, *Wywiad*, *Wyszukiwanie*, *Prezentacja wyników*, *Refinement*, *Zakończona*). Dla wybranych operacji (np. flow rerankingu oraz przetwarzania profilu społecznościowego) przygotowano także diagramy aktywności, które obrazują równoległe gałęzie przetwarzania (scraping wielu sklepów jednocześnie).

10.3 BPMN

Proces biznesowy od momentu wejścia użytkownika na stronę (landing page) do zakończenia sesji wyszukiwania został odwzorowany w notacji BPMN (Rysunek 10.3). Diagram przedstawia m.in. decyzje użytkownika (kontynuować wywiad vs. przejść od razu do wyników), punkty integracji z zewnętrznymi API oraz miejsca potencjalnych błędów (np. brak odpowiedzi od dostawcy), wraz z obsługą wyjątków.



Sformalizowane procesy biznesowe i diagramy aktywności stały się bezpośrednim wsadem do etapu projektowania interfejsu użytkownika.

11 Prototypowanie interfejsu

Na podstawie powyższych modeli zaprojektowano interfejs użytkownika, który prowadzi odbiorcę przez najważniejsze przepływy w możliwie naturalny i zrozumiały sposób.

Interfejs użytkownika powstawał iteracyjnie, w ścisłym powiązaniu z modelem wymagań i architektury. W narzędziu Figma przygotowano prototypy widoków, które następnie przekładano na komponenty React (Shaden UI + Tailwind), zachowując spójność z przepływami zdefiniowanymi na diagramach C4, UML i BPMN.

Poniżej przedstawiono pełne flow użytkownika – od wejścia na stronę, przez konfigurację wyszukiwania i wywiad z chatbotem, aż do otrzymania spersonalizowanych rekomendacji oraz zarządzania zapisanymi produktami.

11.1 Krok 1: Strona główna (Landing Page)

Użytkownik trafia na stronę główną (Rysunek 12a), która w prosty sposób przedstawia wartość aplikacji: „*Znajdź idealny prezent w kilka minut*”. Strona składa się z kilku sekcji:

- **Hero** – główny baner z logo i przyciskiem CTA „Rozpocznij”,
- **Jak to działa** – 4 karty opisujące proces (linki społecznościowe → rozmowa z AI → wyszukiwanie → filtrowanie),
- **Zespół** – informacje o twórcach aplikacji,
- **FAQ** – odpowiedzi na najczęściej zadawane pytania.

Na tym etapie użytkownik może zalogować się przez Google OAuth, co umożliwia zapisywanie historii sesji i ulubionych produktów.

11.2 Krok 2: Formularz wyszukiwania

Po kliknięciu przycisku „Rozpocznij” użytkownik przechodzi do formularza konfiguracji wyszukiwania (Rysunek 12b). Formularz składa się z trzech sekcji:

a) Linki do mediów społecznościowych (opcjonalne) – użytkownik może podać adresy profili Instagram, X (Twitter) lub TikTok obdarowywanej osoby. System wykorzysta publiczne dane do lepszego zrozumienia zainteresowań odbiorcy.

b) Wybór okazji – siatka 4 przycisków pozwala określić kontekst prezentu: Urodziny, Rocznica, Święta, Tak po prostu.

c) Budżet – użytkownik wybiera przedział cenowy (do 50 zł, 50–100 zł, 100–200 zł) lub podaje własny zakres.

AI Present Finder

Znajdź idealne pomysły na prezenty dla bliskich dzięki spersonalizowanym rekomendacjom napędzanym przez AI.

Rozpocznij



Jak to działa -

(a) Strona główna (landing page).

Znajdź idealny prezent

Opowiedz nam o tej osobie

Wpisz adres URL jej profilu w mediach społecznościowych. Obsługujemy Instagram, X i TikTok.

 instagram.com/username

 x.com/username

 tiktok.com/@username

Detale

Jaka jest okazja?



Urodziny



Rocznica

Znajdź pomysły na prezenty



Szukaj



Zapisane



Historia



Profil

Rysunek 12: Początek flow użytkownika: strona główna i formularz konfiguracji wyszukiwania.

Jeśli użytkownik wcześniej szukał prezentu dla tej samej osoby, system zaproponuje wczytanie zapisanego profilu, co przyspiesza proces konfiguracji.

11.3 Krok 3: Wywiad z chatbotem

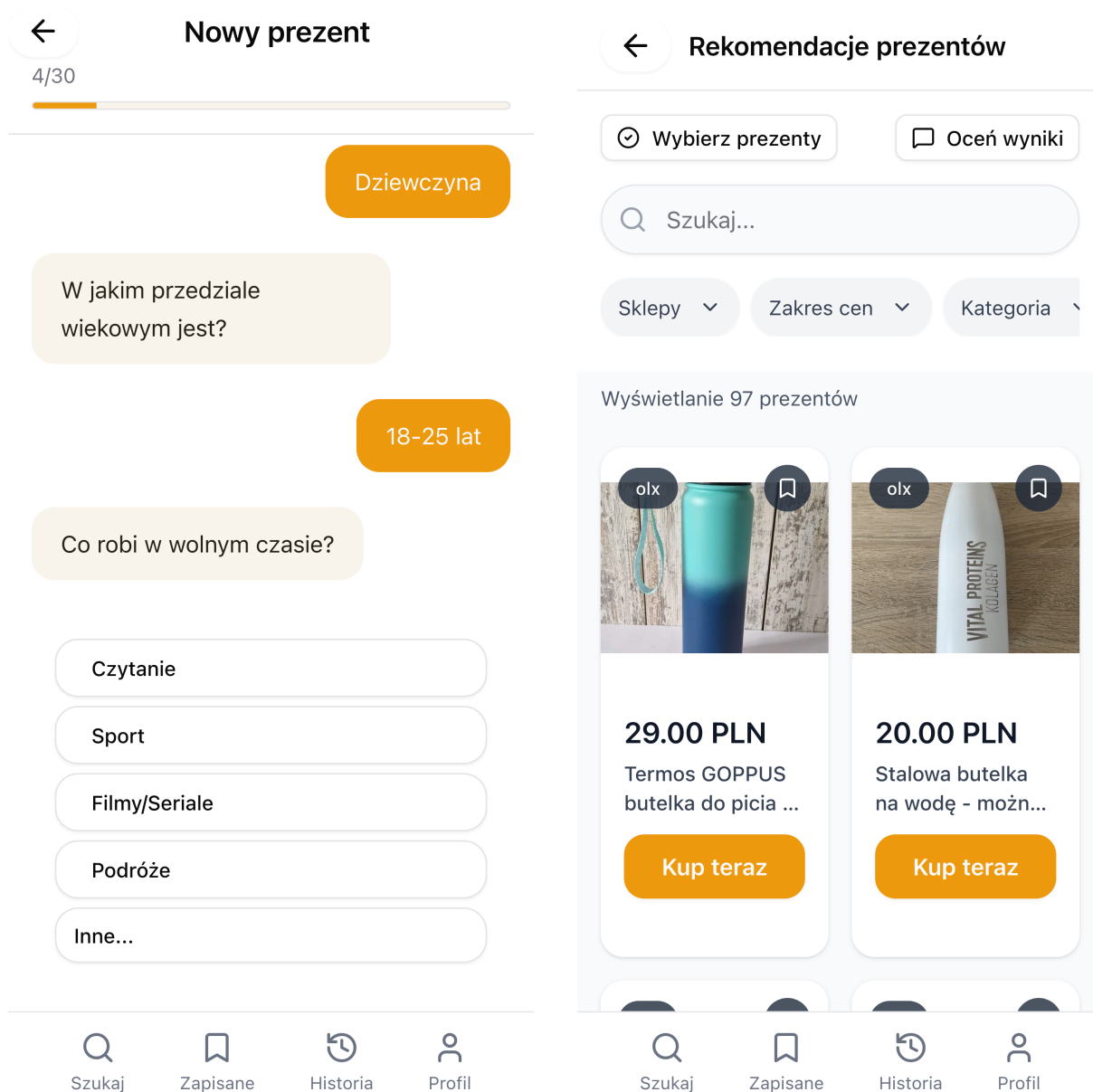
Po wysłaniu formularza użytkownik przechodzi do interfejsu chatu (Rysunek 13a). Agent AI prowadzi krótki wywiad, dopytując o szczegóły dotyczące relacji z obdarowywaną osobą, zainteresowań i hobby odbiorcy oraz preferencji stylistycznych.

Chatbot oferuje sugerowane odpowiedzi (przyciski), ale użytkownik może też wpisać własną odpowiedź. Pasek postępu w nagłówku informuje o etapie wywiadu. Cała rozmowa trwa średnio 2–3 minuty i składa się z ok. 8–12 pytań.

11.4 Krok 4: Lista rekomendacji

Po zakończeniu wywiadu system analizuje zebrane dane, generuje abstrakcyjne pomysły na prezenty, a następnie wyszukuje konkretne oferty w czterech serwisach e-commerce (OLX, Allegro, Amazon, eBay).

Wyniki są prezentowane w formie przejrzystej siatki kart (Rysunek 13b), gdzie każda oferta zawiera zdjęcie produktu, badge z nazwą platformy, tytuł, cenę oraz przycisk „Kup teraz”. Górna belka oferuje rozbudowane opcje filtrowania oraz tryby doprecyzowania i oceniania.



(a) Interfejs chatu z AI.

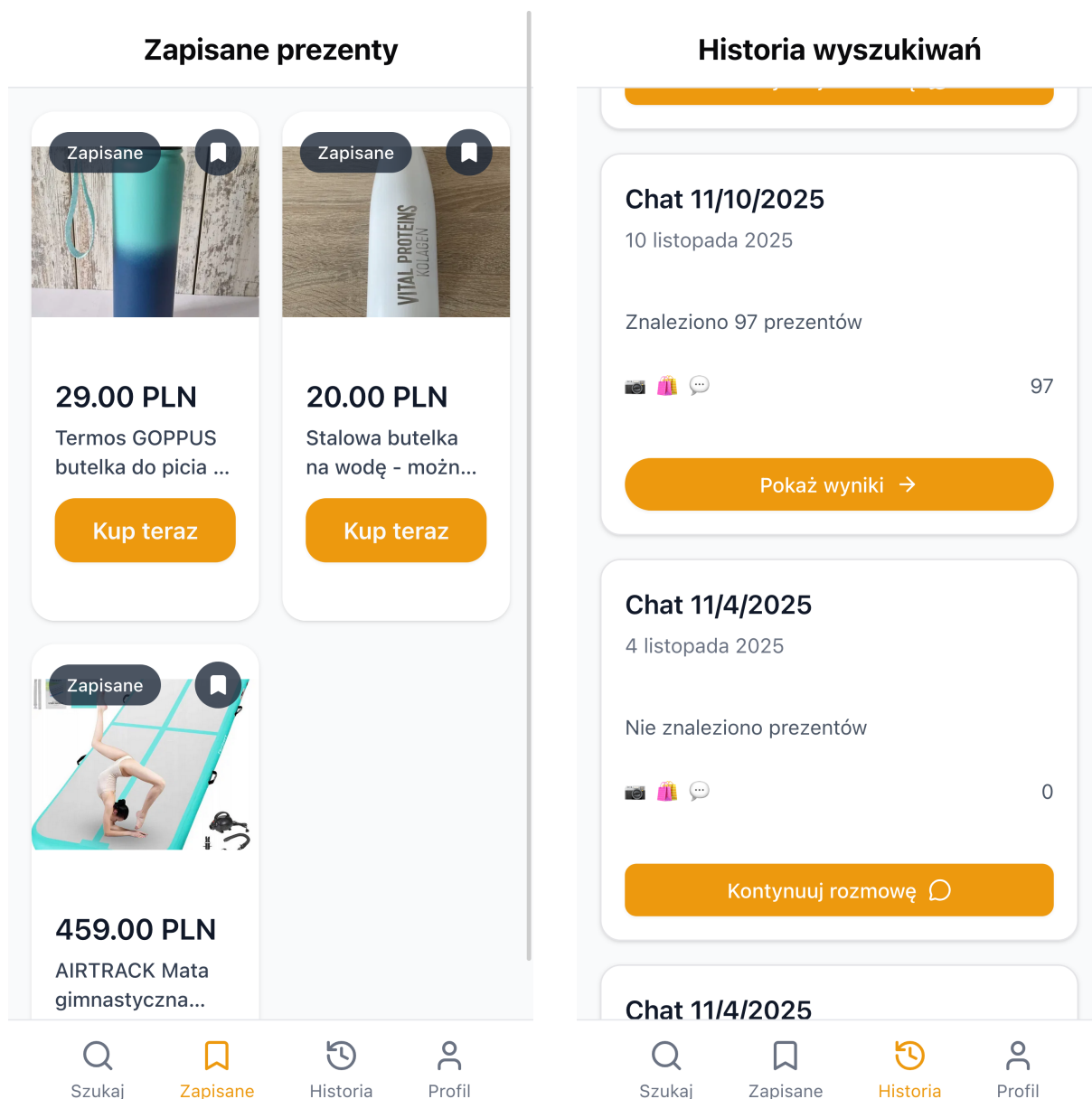
(b) Lista rekomendacji prezentów.

Rysunek 13: Główny flow: wywiad z chatbotem i wynikowa lista rekomendacji.

11.5 Krok 5: Nawigacja i zarządzanie

Dolna belka nawigacyjna (widoczna na wszystkich ekranach po zalogowaniu) zapewnia szybki dostęp do czterech głównych sekcji aplikacji:

- a) **Szukaj** – powrót do formularza nowego wyszukiwania.
- b) **Zapisane** (Rysunek 14a) – lista ulubionych produktów oznaczonych ikoną zakładki podczas przeglądania rekomendacji.
- c) **Historia** (Rysunek 14b) – archiwum poprzednich sesji wyszukiwania z możliwością powrotu do wyników lub kontynuacji niedokończonych rozmów.



(a) Zapisane (ulubione) produkty.

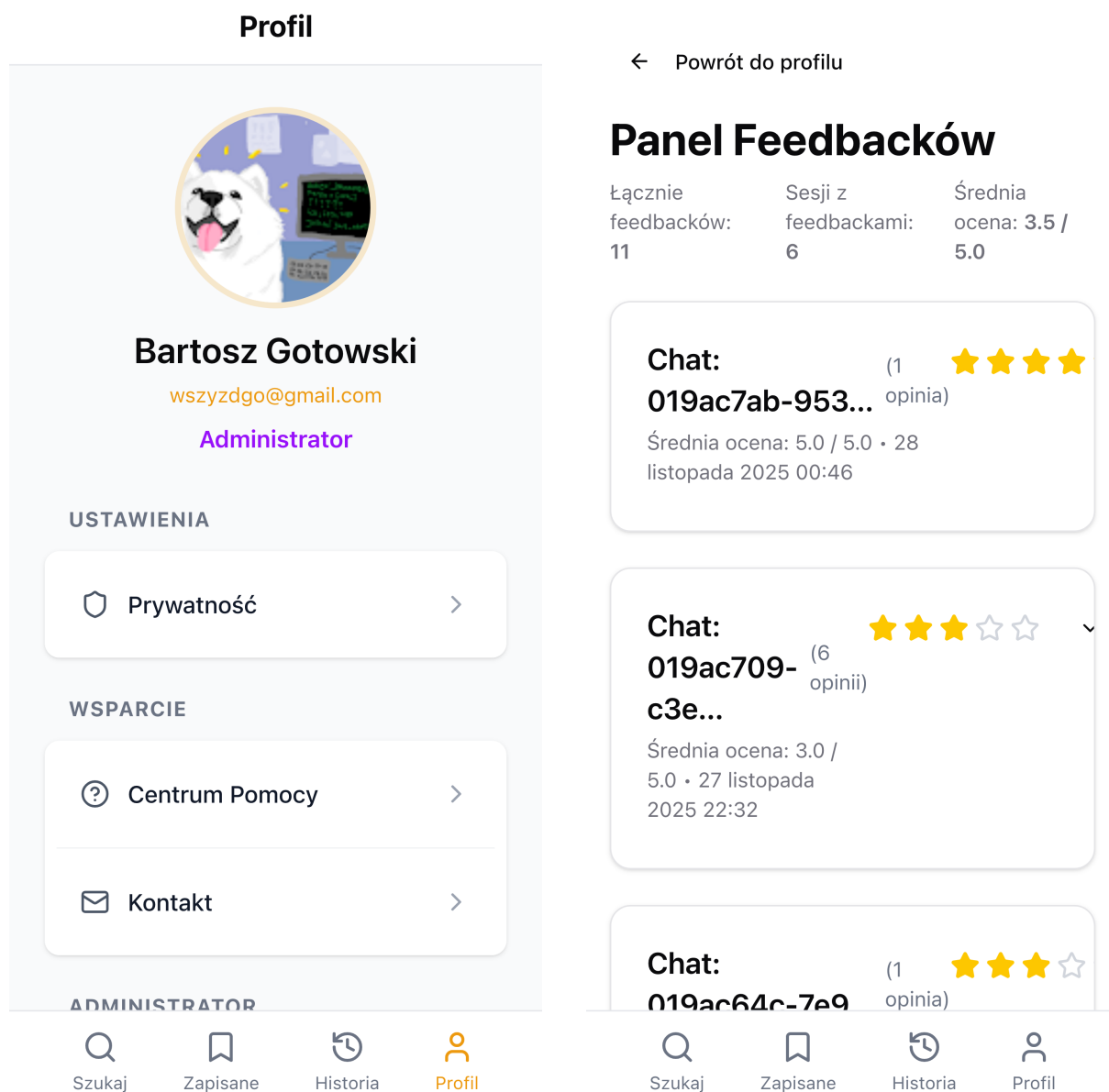
(b) Historia sesji wyszukiwania.

Rysunek 14: Zarządzanie danymi: zapisane produkty i historia sesji.

d) **Profil** (Rysunek 15a) – zarządzanie kontem użytkownika zawierające zdjęcie, dane, ustawienia konta oraz przycisk wylogowania. Administratorzy mają dodatkowo dostęp do panelu feedbacków.

11.6 Panel administracyjny

Użytkownicy z rolą administratora mają dostęp do panelu feedbacków (Rysunek 15b), gdzie mogą przeglądać wszystkie opinie użytkowników pogrupowane według sesji. Panel wyświetla statystyki: łączną liczbę feedbacków, liczbę sesji z opiniami oraz średnią ocenę.



(a) Panel profilu użytkownika.

(b) Panel administracyjny – feedbacki.

Rysunek 15: Profil użytkownika i panel administracyjny.

Zatwierdzone projekty interfejsu oraz architektura systemu pozwoliły na przejście do fazy implementacji, w której zrealizowano kluczowe mechanizmy, w tym algorytmy AI.

12 Implementacja

12.1 Algorytm Rerankingu z Weryfikacją (XAI)

Kluczowym elementem systemu jest moduł rerankingu, który rozwiązuje problem "halucynacji" wyszukiwarek. Tradycyjne wyszukiwanie oparte na słowach kluczowych często zwraca wyniki nieadekwatne (np. akcesoria do konsoli zamiast konsoli). Zaimplementowany algorytm wykorzystuje model LLM (Gemini 2.5) do oceny semantycznej każdego produktu. Dla każdej oferty model generuje:

- Ocenę punktową (0-10) zgodności z profilem odbiorcy.
- Tekstowe uzasadnienie decyzji (np. "Odrzucono: Produkt to część zamienna, a nie gotowy prezent").

Pozwala to na odfiltrowanie ok. 40% szumu informacyjnego.

Poniżej przedstawiono fragment systemu promptów definiującego kryteria oceny produktów:

Jak pokazano w Listingu 1, kryteria oceny produktów są zapisane w postaci czytelnego dla człowieka systemu promptów, który wymusza wyjaśnialne (XAI) decyzje modelu.

Listing 1: Fragment systemu promptów dla rerankingu produktów

```
1 <criteria>
2   <relevance priority="highest">
3     Czy produkt odpowiada zainteresowaniom i hobby odbiorcy?
4   </relevance>
5   <lifestyle_fit priority="high">
6     Czy produkt pasuje do stylu życia i codziennej rutyny?
7   </lifestyle_fit>
8   <preferences priority="high">
9     Czy produkt jest zgodny z preferencjami estetycznymi?
10  </preferences>
11  <occasion priority="medium">
12    Czy produkt jest odpowiedni dla danej okazji?
13  </occasion>
14 </criteria>
15
16 <scoring>
17   1-3: Slabe dopasowanie - produkt nie pasuje do profilu
18   4-6: Srednie dopasowanie - moze pasowac, ale nie jest idealny
19   7-8: Dobrze dopasowanie - produkt dobrze pasuje do profilu
20   9-10: Doskonale dopasowanie - idealnie pasuje do profilu i okazji
21
22   Wyniki ponizej 5 nie beda widoczne uzytkownikowi.
23 </scoring>
```

12.2 Orkiestracja zdarzeń w architekturze CQRS

Centralnym elementem systemu jest handler `StartProcessingCommand`, który orkiestruje rozpoczęcie całego procesu wyszukiwania prezentu. Po otrzymaniu żądania od użytkownika tworzy sesję chatu w bazie danych, a następnie równolegle publikuje dwa zdarzenia do kolejki RabbitMQ: jedno uruchamia analizę OSINT profili społecznościowych, drugie – wywiad z chatbotem.

Listing 2 przedstawia uproszczoną implementację tego handlera, podkreślając zasadę: najpierw utrwalenie sesji w bazie, dopiero później emisja zdarzeń do innych mikroserwisów.

Listing 2: Orkiestracja zdarzeń przy rozpoczęciu wyszukiwania prezentu

```
1 @CommandHandler(StartProcessingCommand)
2 export class StartProcessingCommandHandler {
3   constructor(
4     @Inject("STALKING_ANALYZE_REQUESTED_EVENT")
5     private readonly stalkingEventBus: ClientProxy,
6     @Inject("CHAT_START_INTERVIEW_EVENT")
7     private readonly chatEventBus: ClientProxy,
8     private readonly chatRepository: IChatRepository,
9   ) {}
10
11   async execute(command: StartProcessingCommand) {
12     // 1. Najpierw tworzymy sesję w bazie danych
13     const chat = await this.chatRepository.create({
14       chatId: analyzeRequestedDto.chatId,
15       chatName: `Chat ${new Date().toLocaleDateString()}`,
16       userId,
17     });
18
19     // 2. Dopiero po utrwaleniu emitujemy zdarzenia równoległe
20     this.stalkingEventBus.emit(
21       StalkingAnalyzeRequestedEvent.name, stalkingEvent
22     );
23     this.chatEventBus.emit(
24       ChatStartInterviewEvent.name, interviewEvent
25     );
26   }
27 }
```

12.3 Strukturyzacja rozmowy przez Tool Calling

Moduł chatu wykorzystuje mechanizm *tool calling* do wymuszenia strukturyzowanej odpowiedzi od modelu LLM. Zamiast swobodnego tekstu, AI musi wywołać jedno z trzech narzędzi: zadać pytanie z sugerowanymi odpowiedziami, zakończy wywiad z wygenerowanym profilem, lub oznaczyć nieodpowiednie zapytanie.

Listing 3 pokazuje definicję tych narzędzi wraz z odpowiadającymi im schematami Zod, co zapewnia typowaną i przewidywalną współpracę pomiędzy LLM a resztą systemu.

Listing 3: Definicja narzędzi AI dla strukturyzacji wywiadu

```
1 export const tools = {
2   ask_a_question_with_answer_suggestions: tool({
3     description: "Zadaj pytanie z 4 sugerowanymi odpowiedziami",
4     inputSchema: z.strictObject({
5       question: z.string().min(1),
6       potentialAnswers: potencialAnswersSchema,
7     }),
8   }),
9
10  end_conversation: tool({
11    description: "Zakończ rozmowę z wygenerowanym profilem",
12    inputSchema: z.object({
```

```

13     output: endConversationOutputSchema, // profil odbiorcy
14   }},
15 },
16
17 flag_inappropriate_request: tool({
18   description: "Oznacz nieetyczne lub nielegalne zapytanie",
19   inputSchema: z.object({
20     reason: z.string(),
21   }),
22 },
23 } as const;

```

Dzięki temu podejściu każda odpowiedź AI jest typowana i walidowana schematem Zod, co eliminuje problemy z parsowaniem i zapewnia spójność przepływu danych między mikroserwisami.

12.4 Inżynieria promptów (Prompt Engineering)

Jednym z najważniejszych aspektów implementacji systemu AI Present Finder była iteracyjna optymalizacja promptów – instrukcji przekazywanych modelom językowym. W ciągu 80 dni rozwoju projektu (wrzesień–listopad 2025) przeprowadzono ponad 25 iteracji promptów, co przełożyło się na znaczącą poprawę jakości generowanych rekomendacji.

Ewolucja promptów. System wykorzystuje 5 specjalizowanych promptów, z których każdy przeszedł własną ścieżkę optymalizacji:

Prompt	Iteracje	Kluczowe zmiany
Chat (wywiad)	12	Format XML, min. 30 pytań, 3. osoba, podejście produktowe
Refinement	2	Doprecyzowanie po wyborze produktów (3–5 pytań)
Stalking (OSINT)	3	Ekstrakcja faktów z profili społecznościowych
Gift Ideas	5	Generowanie zapytań dla 5 platform, max 5 wyrazów
Reranking	4	Scoring 1–10, filtrowanie ≥ 5 , batching

Tabela 2: Przegląd promptów i liczba iteracji w projekcie

Kluczowe problemy i rozwiązania. Podczas rozwoju systemu zidentyfikowano i rozwiązano szereg problemów związanych z zachowaniem modeli LLM:

- **Za krótkie rozmowy:** Początkowy prompt generował tylko 10 pytań, co dawało niewystarczające dane. Rozwiązanie: dodanie licznika pytań i wymuszenie minimum 30 pytań przed zakończeniem wywiadu.
- **Pytania w 2. osobie:** Model pytał „Czy lubisz gotować?” zamiast „Czy ON/ONA lubi gotować?”. Rozwiązanie: explicite reguła w prompcie wymuszająca 3. osobę.
- **Zbyt abstrakcyjne pytania:** Pytania typu „Jaki styl preferuje?” nie prowadziły do konkretnych kategorii produktów. Rozwiązanie: podejście produktowe – „Czy ma dobre słuchawki?” zamiast „Jaki rodzaj muzyki lubi?”.

- **Brak różnorodności:** Model zawsze zaczynał od pracy zawodowej. Rozwiązanie: lista 16 obszarów eksploracji z wymogiem pokrycia min. 5 różnych.
- **Drażnienie po „nie wiem”:** Model kontynuował ten sam wątek mimo braku wiedzy użytkownika. Rozwiązanie: reguła natychmiastowej zmiany obszaru po odpowiedzi „nie wiem”.
- **Brakujące tool calls:** Model czasami odpowiadał tekstem zamiast wywołać narzędzie. Rozwiązanie: mechanizm auto-retry z przypomnieniem w kolejnej wiadomości.

Struktura prompta wywiadu. Największy prompt (Chat) liczy ponad 700 linii i wykorzystuje format XML dla lepszej czytelności przez model. Listing 4 przedstawia uproszczoną strukturę tego prompta.

Listing 4: Struktura prompta wywiadu z użytkownikiem (uproszczony)

```

1 <system>
2   <role>Jestes Doradca Prezentowym - prowadzisz rozmowe
3     (15-30 pytan) aby poznac obdarowywanego.</role>
4
5   <context>
6     <occasion>${occasion}</occasion>
7     <conversation_progress>
8       <current_question_number>${questionCount}</current_question_number>
9       <status>MUSISZ ZADAC PRZYNAJMNIEJ ${30 - questionCount}
10      PYTAN WIECEJ!!!</status>
11     </conversation_progress>
12   </context>
13
14   <critical_rules>
15     <rule id="1">JEDNO pytanie na raz, PROSTE, konkretne</rule>
16     <rule id="2">TRZECIA osoba (on/ona) - NIGDY druga osoba</rule>
17     <rule id="3">Pytaj PRODUKTOWO, nie abstrakcyjnie</rule>
18     <rule id="5">Eksploruj MINIMUM 5 ROZNYCH obszarow zycia</rule>
19     <rule id="6">"Nie wiem" = NATYCHMIAST zmien watek</rule>
20   </critical_rules>
21
22   <exploration_leads>
23     PRACA, DOM, HOBBY, KULINARIA, TECHNOLOGIA, ZDROWIE,
24     PODROZE, MUZYKA, ZWIERZETA, SZTUKA, GAMING, FOTOGRAFIA...
25   </exploration_leads>
26
27   <tools>
28     <tool name="ask_a_question_with_answer_suggestions"/>
29     <tool name="end_conversation"/>
30     <tool name="flag_inappropriate_request"/>
31   </tools>
32 </system>

```

Wyniki optymalizacji. Iteracyjne doskonalenie promptów przyniosło wymierne rezultaty:

- Liczba pytań w wywiadzie: 10 → 30 (3× więcej danych o odbiorcy),
- Obszary eksploracji: 6 → 16+ kategorii życia,

- Sukces tool calls: $\sim 60\% \rightarrow \sim 95\%$ (dzięki auto-retry),
- Trafność pierwszej rekomendacji: wzrost do 64% (First Hit Accuracy).

Kluczowym wnioskiem z procesu prompt engineeringu jest znaczenie **konkretnych przykładów i explicite zakazów** w instrukcjach. Model lepiej rozumie „NIE pytaj o budżet” niż „skup się na zainteresowaniach”. Podobnie, podanie 2–3 przykładowych rozmów w prompcie drastycznie poprawia jakość generowanych pytań.

12.5 Równoległa Agregacja Ofert

Ze względu na limity API i czas odpowiedzi serwisów e-commerce, zaimplementowano mechanizm równoległego pobierania danych. **Fetch Microservice** może być uruchamiany w wielu instancjach, z których każda obsługuje innego dostawcę (OLX, Allegro, Amazon). Wyniki są asynchronicznie przesyłane do głównego strumienia zdarzeń, co pozwala na prezentowanie pierwszych ofert użytkownikowi bez czekania na zakończenie przeszukiwania wszystkich źródeł.

Zaimplementowany system został poddany weryfikacji poprzez szereg testów, których przebieg i wyniki opisano poniżej.

13 Testy

13.1 Planowanie testów

Strategia testowania zakładała kilka poziomów:

- testy jednostkowe wybranych fragmentów logiki (np. parserów danych, prostych serwisów domenowych),
- testy integracyjne mikroservisów (sprawdzenie poprawności kontraktów zdarzeń i komunikacji przez RabbitMQ),
- testy end-to-end podstawowego flow użytkownika (logowanie, wywiad, otrzymanie rekomendacji),
- testy wydajnościowe i odporności na błędy API (limity zapytań, czas odpowiedzi).

Do testów jednostkowych i integracyjnych wykorzystano ekosystem NestJS (Jest), natomiast testy E2E przeprowadzono z użyciem narzędzi takich jak Playwright lub Cypress (uruchamianych ręcznie dla kluczowych scenariuszy).

13.2 Testy z użytkownikami

Kluczowym elementem weryfikacji jakości systemu były beta-testy z udziałem rzeczywistych użytkowników. Zebrano opinie dotyczące trafności rekomendacji (oceny gwiazdkowe, komentarze tekstowe) oraz wrażeń z korzystania z interfejsu (czas oczekiwania, przejrzystość, intuicyjność). Dane te zostały wykorzystane zarówno w sekcji wyników, jak i przy planowaniu dalszego rozwoju (np. dodanie pamięci negatywnej, lepsze dopytywanie o budżet).

Jednym z głównych ryzyk, które miały wychwycić testy z użytkownikami, była odporność systemu na częściowe awarie integracji z serwisami e-commerce. W kilku sesjach

celowo wyłączono jedną z instancji **Fetch Microservice** (np. odpowiedzialną za Allegro), aby sprawdzić, czy użytkownik nadal otrzyma sensowny zestaw propozycji z pozostałych źródeł i czy interfejs poprawnie komunikuje ograniczoną dostępność wyników.

W ramach testów szczególną uwagę zwrócono na scenariusz użytkownika szukającego prezentu dla fotografa-amatora. System zaproponował m.in. analogowy aparat z OLX, zestaw akcesoriów do aparatu oraz kurs fotografii online, a użytkownik ocenił całą sesję na 5/5 gwiazdek, podkreślając, że sam nie wpadłby na część tych pomysłów.

Pozytywne wyniki testów pozwoliły na przeprowadzenie badań z użytkownikami końcowymi, których rezultaty przedstawiono w ostatniej sekcji.

14 Wyniki i analiza badań

System został poddany testom w środowisku produkcyjnym (pilotaż, listopad 2025). W analizie wzięto pod uwagę 36 pełnych sesji zakupowych.

14.1 Efektywność czasowa

Średni czas potrzebny na znalezienie prezentu został zredukowany z rynkowej średniej 15 godzin (rocznie) do **7 minut i 44 sekund** na sesję. W tym czasie system przeprowadza wywiad z użytkownikiem, analizuje profil społecznościowy i przeszukuje cztery platformy e-commerce.

14.2 Jakość rekomendacji

- **Trafność (First Hit Accuracy):** 64% użytkowników uznało pierwszą propozycję za trafną.
- **Redukcja szumu:** Moduł rerankingu odrzuca średnio 40% ofert jako nietrafione lub niskiej jakości (np. części zamienne zamiast produktów).
- **Satysfakcja użytkowników:** Średnia ocena 3.5/5.0. Użytkownicy docenili trafność pomysłów, ale wskazywali na problem polecania rzeczy, które obdarowywany już posiada (brak "pamięci negatywnej" w MVP).

14.3 Przykłady rerankingu AI (Explainable AI)

Moduł rerankingu wykorzystuje model Gemini 2.5 Flash Lite do oceny każdego produktu w skali 1–10 wraz z uzasadnieniem. Na podstawie danych z produkcji (2512 produktów, 2380 z oceną) przeanalizowano rozkład ocen:

Zakres ocen	Liczba produktów	Procent
Ocena 1–3 (odfiltrowane)	391	16%
Ocena 4–5 (słabe dopasowanie)	267	11%
Ocena 6–7 (dobre dopasowanie)	779	33%
Ocena 8–9 (bardzo dobre)	899	38%
Ocena 10 (idealne)	44	2%
Średnia ocena	6.38/10	

Tabela 3: Rozkład ocen rerankingu AI (dane produkcyjne, grudzień 2025)

Produkty z oceną 10 (idealne dopasowanie)

Tytuł	Cena	Ocena	Uzasadnienie AI
Herman Miller Aeron C full pakiet	3 800 zł	10	Najwyższej klasy fotel biurowy, znany z niezrównanej ergonomii i komfortu, idealnie wpisuje się w potrzeby osoby ceniącej wygodę siedzenia.
Fotel gamingowy Diablo X-Horn	500 zł	10	Wygodny i stylowy fotel gamingowy, zapewniający najwyższy komfort i funkcjonalność. Idealnie pasuje do potrzeb odbiorcy szukającego ergonomicznego fotela.
Fotel Secret Lab jak NOWY + akcesoria	1 599 zł	10	Fotel gamingowy Secret Lab jak nowy, z dodatkowymi akcesoriami i w świetnej cenie, jest idealnym prezentem.

Produkty z oceną 1 (odfiltrowane jako niepasujące)

Tytuł	Cena	Ocena	Uzasadnienie AI
Duża kuchenka dla dzieci mega zestaw	189 zł	1	Duża kuchenka dla dzieci jest całkowicie nieodpowiednia dla odbiorcy w wieku 18–25 lat i nie ma związku z jego zainteresowaniami.
Koci Domek Gabi z figurkami	249 zł	1	Produkt jest zabawką przeznaczoną dla małych dzieci, co całkowicie mija się z zainteresowaniami i wiekiem odbiorcy.
Kuchnia drewniana IKEA dla dzieci	120 zł	1	Zabawka przeznaczona dla dzieci 1–3 lata, całkowicie nieodpowiednia dla dorosłego odbiorcy.

Reranking skutecznie eliminuje produkty nietrafione kontekstowo (zabawki dla dzieci przy szukaniu prezentów dla dorosłych) i przyznaje wysokie oceny produktom dokładnie odpowiadającym profilowi odbiorcy. System odfiltrował 27% produktów (oceny 1–3) jako niepasujące.

14.4 Feedback użytkowników

W ramach testów pilotażowych zebrano 11 opinii od użytkowników. Poniżej przedstawiono reprezentatywne cytaty:

Opinie pozytywne

- "Trafiony w sedno jak na tak niedługą i przyjemną rozmowę." (Ocena: 5/5)
- "Szybko, sprawnie, widać że system próbuje zrozumieć kontekst. Propozycje były sensowne i w budżecie." (Ocena: 4/5)
- "Świetny pomysł na aplikację! Wyniki były zaskakująco trafne." (Ocena: 4/5)

Opinie krytyczne i sugestie

- *"Większość prezentów miała wspólne cechy z prezentami które były kupione w przeszłości — system polecał rzeczy, które osoba już ma."* (Ocena: 3/5) — wskazuje na potrzebę "pamięci negatywnej".
- *"OLX przytłaczające, brak wyników z Allegro. Rozmowa była zbyt długa."* (Ocena: 2/5) — sugestia balansowania źródeł (brak wyników wynikał z użycia środowiska Allegro Sandbox) i skrócenia wywiadu.

Główne wnioski z feedbacku:

1. 64% użytkowników uznało pierwszy wynik za trafny
2. Najczęstszy problem: polecenie przedmiotów już posiadanych przez obdarowywanego
3. Użytkownicy docenili szybkość działania i naturalność konwersacji

14.5 Stabilność

System osiągnął 100% dostępności (uptime) podczas testów, przy minimalnym zużyciu zasobów (<800MB RAM dla całego klastra), co potwierdza poprawność założeń architektonicznych.